

Ask My Cohort — Build Day Operating Plan

Team **tech** · Databricks Campus Hackathon, BMSCE · Build day 1 Sep 2026 · 09:00–18:00, nine hours · For all four members

Four people, nine hours, one Genie agent over governed data. This document assigns every hour to a named person, fixes the schema so nobody blocks on a naming decision, and provides the exact SQL and prompts to paste. Each member reads their own section first, then section 3.

1 · Who owns what

Person	Owns	Definition of done by 16:30
Ojash	Unity Catalog governance, Genie space, driving the demo	Three roles log in and get three different answers to the identical question. The Genie space has comments, instructions, and six verified example questions.
Aditya	The entire data and ML lane, alone	Three gold tables exist with exactly the columns in section 2, populated, with <code>date <= 42</code> applied once in silver. Two MLflow runs logged.
Satyam	Deck, demo script, submission, the clock	Deck done by 14:30. Demo script timed to 3:00. Submission filed by 17:00. The hard freeze called out loud at 16:30.
Kirtee	QA, evidence, jury Q&A	25 advisor questions tested against Genie with pass/fail logged. Screenshots of all three role views captured. Q&A cue cards ready.

The Aditya boundary. Ojash and Aditya never edit the same file. Aditya owns every notebook that *writes* tables. Ojash owns every statement that *governs* tables. The dependency runs one way: Aditya produces `campus.gold.*`, Ojash attaches filters and masks to them. If a gold column is wrong, Ojash tells Aditya and Aditya fixes it — nobody opens someone else's notebook. This boundary is the only thing standing between the team and a merge argument at hour nine.

2 · Schema contract — frozen at hour 0

Everyone codes against these names. No renaming after hour 1. This page gets printed or pinned in the group chat.

```
Catalog: campus
Schemas: campus.bronze    campus.silver    campus.gold    campus.ops
```

Bronze — one table per source file, nothing cleaned

```
campus.bronze.student_info      campus.bronze.assessments
campus.bronze.student_vle      campus.bronze.student_registration
campus.bronze.student_assessment campus.bronze.courses
campus.bronze.vle              campus.bronze.attendance_synth
```

Silver — one row per student per module per week

```
campus.silver.student_week
student_id STRING, code_module STRING, code_presentation STRING,
week INT, clicks INT, active_days INT, distinct_resources INT,
submissions INT, avg_score DOUBLE, days_late_avg DOUBLE,
sessions_held INT, sessions_attended INT, is_at_risk INT
-- WHERE date <= 42 applied HERE and nowhere else
```

Gold — the three tables Genie answers from

```
campus.gold.risk_signals
student_id, student_name, code_module, code_presentation,
advisor_id, department, risk_score DOUBLE, risk_band STRING,
top_factor_1, top_factor_2, top_factor_3, scored_at TIMESTAMP

campus.gold.session_forecasts
code_module, code_presentation, session_date DATE,
expected_headcount INT, lower_bound INT, upper_bound INT, model_version

campus.gold.attendance_buffers
student_id, student_name, code_module, advisor_id, department,
sessions_held INT, sessions_attended INT, attendance_pct DOUBLE,
threshold_pct DOUBLE, sessions_remaining INT, sessions_missable INT,
buffer_band STRING, -- low | medium | high | unavoidable
cost_of_missing_next STRING
```

Ops — the role table the governance depends on

```
campus.ops.role_map
user_email STRING, -- exact Databricks login email, lowercase
role STRING,       -- advisor | dean | student | admin
```

```
advisor_id STRING,    -- populated for advisors, NULL otherwise
student_id STRING,    -- populated for students, NULL otherwise
department STRING
```

Why `role_map` and not `is_account_group_member()` . The group function is the textbook answer, but on Free Edition the account console may be restricted, and the function fails *silently* — the filter evaluates false for everyone and the demo shows empty tables. `role_map` plus `current_user()` works with four ordinary logins and nothing else. The team builds it this way. If groups happen to work, that is a bonus mentioned verbally, not a dependency shipped.

3 · The nine hours

The organisers' portal gives the build window as **09:00 to 18:00 on 1 September — nine hours, not twelve**. The submission phase closes at 18:00. Every timing below is real clock time and the plan has been compressed to fit; three hours came out, and section 3b says exactly which three.

Clock	Ojash	Aditya	Satyam	Kirtee
6:20 -9:00	Gate, registration, setup. Not build time — see the note below before touching anything. Every laptop logs into Databricks and the tabs stay open. Power and wifi confirmed at every seat. The demo laptop is identified. Everyone eats.			
9:00 -9:30	Catalog and four schemas created. <code>ops.role_map</code> created and populated with four real emails.	OULAD upload started — <code>studentVle</code> first.	Every login email collected and posted. Schema contract pinned in the chat.	Confirms all four accounts verified. Opens the QA sheet.
9:30 -11:00	Grant chain to all four users. Toy row filter re-tested in the real workspace, on real logins.	Synthetic attendance generator. Hard stop at 10:30 — 60 minutes, not 90.	Deck slides 1-4.	Finishes the 25 questions. Runs cells Aditya hands over.
11:00 -12:30	Genie space created early. Table and column comments written against the schema contract, before gold exists.	Bronze ingest, then silver join. <code>date <= 42</code> applied here, once.	Deck slides 5-6. Lunch rotation so the room is never empty.	Screenshot and screen-recording setup verified on the demo machine.
12:30 -14:00	Genie instructions block. The six example questions drafted with SQL.	Both models. Logistic regression first and logged before anything fancier is attempted. Demographic check at 13:00.	Deck content finalised.	Tests questions against silver as a stand-in for gold.
14:00 -15:00	Genie space attached to the real gold tables. Example questions verified against live data.	Gold tables materialised by 15:00. Messages the group: "gold is ready, schema matches contract."	Deck frozen at 14:30. Rehearses narration solo.	Full test pass against gold. Failures reported as exact question text.
15:00 -16:00	Row filters and column masks applied and tested per role. Warehouse restarted if anything looks stale.	Support only. No new features. Fixes what the tests break.	Script timed with a stopwatch to 3:00.	The three role screenshots captured. These are the insurance.
16:00 -16:30	Last fixes only. Nothing new starts in this window. Anything unfinished at 16:30 is abandoned, not rushed.			
16:30	HARD FREEZE. Satyam calls it out loud. This is now 90 minutes before the portal closes, not two hours. It is the least negotiable line in the document.			
16:30 -17:00	Demo rehearsed end to end twice, on the demo machine, on the venue network. One clean run screen-recorded and saved locally.			
17:00 -17:30	Satyam submits. An hour before close, not ten minutes. The form has thrown 422 errors on fields it does not render.			
17:30 -18:00	Buffer for whatever broke. Cue cards read aloud once. Nothing is being built.			

About the 6:20 to 9:00 window. That is two hours and forty minutes of sitting time before the clock starts, and it is tempting to treat it as free build time. Someone asks an organiser first, in person, at registration. Most events allow account setup and data upload before the start but prohibit building the project itself, and an elimination round is the wrong place to find out which rule applies here. Uploading 464 MB of a public dataset and logging into a workspace is almost certainly fine and is where that window is genuinely worth spending. Writing project code before 09:00 is not worth the risk.

3b · What three hours came out

The original plan was written for twelve hours. Cutting to nine means cutting real work, and it is better done now, on paper, than at 15:00 by whoever is most tired.

Cut	Reasoning
The Databricks App entirely	It was already gated on everything else being green. In nine hours it will not be, and Genie is the product regardless. This is not a loss — the submission already states that no dashboard exists.
The second model attempt	Logistic regression gets logged to MLflow and that is the shipped model. Gradient boosting is attempted only if there is genuinely nothing else to do, which there will not be.
30 minutes of data generation	60 minutes, not 90. It was already described as the least valuable hour of the build.
The slack around Genie	The Genie space now starts at 11:00 against the schema contract rather than at 14:00 against real tables. Comments and instructions can be written before the data exists, because they describe what columns mean, not what is in them. This is the single most important compression in the plan and it only works if the schema contract is genuinely frozen at 09:30.

What did not get cut, and will not be. The governance layer, the Genie space curation, and the freeze. Those three are the submission. If the build falls behind, the risk model is what gets simplified — never the filters, never the comments, never the rehearsal.

4 · The four governance statements — Ojash

The pattern gets tested on a toy table the night before. On build day this is pasting, not thinking.

Setup — run once at hour 0

```
CREATE CATALOG IF NOT EXISTS campus;
CREATE SCHEMA IF NOT EXISTS campus.bronze;
CREATE SCHEMA IF NOT EXISTS campus.silver;
CREATE SCHEMA IF NOT EXISTS campus.gold;
CREATE SCHEMA IF NOT EXISTS campus.ops;

CREATE TABLE IF NOT EXISTS campus.ops.role_map (
  user_email STRING, role STRING, advisor_id STRING,
  student_id STRING, department STRING
);

INSERT INTO campus.ops.role_map VALUES
('ojash@example.com', 'advisor', 'ADV001', NULL, 'Computer Science'),
('aditya@example.com', 'dean', NULL, NULL, 'Computer Science'),
('satyam@example.com', 'student', NULL, 'S100234', 'Computer Science'),
('kirtee@example.com', 'admin', NULL, NULL, NULL);

-- grant chain, repeated per user
GRANT USE CATALOG ON CATALOG campus TO `aditya@example.com`;
GRANT USE SCHEMA ON SCHEMA campus.gold TO `aditya@example.com`;
GRANT SELECT ON TABLE campus.gold.risk_signals TO `aditya@example.com`;
```

Statement 1 — the row filter function

```
CREATE OR REPLACE FUNCTION campus.ops.rf_risk(adv_id STRING, stu_id STRING)
RETURN
EXISTS (SELECT 1 FROM campus.ops.role_map m
  WHERE lower(m.user_email) = lower(current_user())
  AND m.role IN ('admin','dean'))
OR EXISTS (SELECT 1 FROM campus.ops.role_map m
  WHERE lower(m.user_email) = lower(current_user())
  AND m.role = 'advisor' AND m.advisor_id = adv_id)
OR EXISTS (SELECT 1 FROM campus.ops.role_map m
  WHERE lower(m.user_email) = lower(current_user())
  AND m.role = 'student' AND m.student_id = stu_id);
```

Statement 2 — the column mask function

```
CREATE OR REPLACE FUNCTION campus.ops.mask_name(name STRING)
RETURN CASE
  WHEN EXISTS (SELECT 1 FROM campus.ops.role_map m
    WHERE lower(m.user_email) = lower(current_user())
    AND m.role IN ('advisor','admin','student'))
  THEN name
  ELSE 'REDACTED'
END;
```

Statement 3 — attach the filter

```
ALTER TABLE campus.gold.risk_signals
  SET ROW FILTER campus.ops.rf_risk ON (advisor_id, student_id);

ALTER TABLE campus.gold.attendance_buffers
  SET ROW FILTER campus.ops.rf_risk ON (advisor_id, student_id);
```

Statement 4 — attach the mask

```
ALTER TABLE campus.gold.risk_signals
  ALTER COLUMN student_name SET MASK campus.ops.mask_name;

ALTER TABLE campus.gold.attendance_buffers
  ALTER COLUMN student_name SET MASK campus.ops.mask_name;
```

Rollback, in case the demo needs a clean table fast

```
ALTER TABLE campus.gold.risk_signals DROP ROW FILTER;
ALTER TABLE campus.gold.risk_signals ALTER COLUMN student_name DROP MASK;
```

Three things that trip people up here.

- The filter function runs with the definer's rights, so other users do not need `SELECT` on `ops.role_map` — but they may need `EXECUTE` on the function. This gets tested as a second user the night before, not on build day.
- Permission and filter changes cache. If a role suddenly sees the wrong rows, the SQL warehouse gets restarted before anything else is debugged. This has eaten entire afternoons.
- `current_user()` returns the login email, and case mismatches silently return zero rows. That is why every comparison above is

wrapped in `lower()` .

5 · The Genie space — the part that decides the demo

Genie's answer quality is almost entirely a function of what is written in the space, not what is in the tables. Hours 6–9 are budgeted for this, and nobody talks Ojash into skipping it.

Only the three gold tables get attached.

Not bronze, not silver. Every extra table is another way for Genie to generate the wrong join on stage.

Every table and every column gets a comment.

```
COMMENT ON TABLE campus.gold.risk_signals IS
  'One row per student per module. Risk of failing or withdrawing, scored from
  the first six weeks of behaviour only. Use this table for any question about
  which students are at risk, struggling, falling behind, or likely to fail.';

COMMENT ON COLUMN campus.gold.risk_signals.risk_band IS
  'Risk category: high, medium, or low. "At risk" means risk_band = ''high''.';

COMMENT ON COLUMN campus.gold.attendance_buffers.sessions_missable IS
  'Number of future sessions this student can still miss and remain above the
  attendance threshold. Zero means the threshold is already unreachable.';
```

Instructions block — pasted into the space's instructions field.

This workspace answers questions from a faculty advisor about student risk and attendance for the current term.

- "At risk" always means risk_band = 'high' in campus.gold.risk_signals.
- "My students" means every row the user can see; do not filter by advisor in SQL, row-level security already handles it.
- Attendance buffer bands are: low (comfortable), medium (shrinking), high (every session matters), unavoidable (threshold unreachable).
- When asked about attendance, always report the cost of missing the next session, never the number of sessions the student is allowed to skip.
- Risk scores come from behaviour in weeks 1 to 6 only.
- Attendance data is synthetic and disclosed as such.
- Never present a risk flag as a decision or a grade. It is a prompt for a human advisor to look closer.

Six example questions with verified SQL.

Ojash writes the SQL, runs it, confirms the shape, then saves it as an example. The six are:

1. Which of my students are at risk this term?
2. Show me the top three factors behind each high-risk student.
3. Which students have the least attendance buffer left?
4. How many students are expected at the next session for CS201?
5. Which students are high risk and also in the unavoidable attendance band?
6. How many at-risk students are there per department?

Question 6 is the governance moment. It must return department aggregates for the dean and named students for the advisor, from identical text.

6 · What each member needs told — send these verbatim

To Aditya

"You own the whole data and ML lane end to end, alone. Nobody else touches your notebooks and you don't touch anyone else's. Your contract with the team is the schema in section 2 — three gold tables, exact column names, populated by 15:00. Three rules that cannot bend: the `date <= 42` filter goes in silver exactly once; no demographic column enters any feature set; the label is binary, Fail plus Withdrawn equals at-risk. Attendance is generated from a causal story, not randomised — independently randomised columns give the model nothing to learn and you'll spend two hours debugging code that isn't broken. If the model hasn't converged by 13:30, ship logistic regression and move on. Message the group the moment gold is ready; don't wait to be asked."

To Satyam

"You own the deck, the demo script, the clock, and the submission. Three things only you can do. First: the deck is frozen at 14:30, so build it against the plan, not against what's currently working — the plan is what we're pitching. Second: you are the timekeeper. At 16:30 you say 'hard freeze' out loud and you mean it, even if someone is mid-fix. We only have nine hours, not twelve, so this matters more than it did. Third: you file the submission at 17:00, a full hour before the portal closes. The form has thrown 422 errors on missing fields before, so leave room for it to fight you. Write the 3-minute script with a stopwatch — 25s user, 45s the ask, 25s the buffer, 55s governance, 30s honesty."

To Kirtee

"You are the only person who will use the product the way a real advisor would, and that makes you the most useful tester the team has. Write 25 plain-English questions a faculty advisor would actually ask — not questions you know our tables can answer. From hour 6, run every one against Genie and log pass or fail in a shared sheet. When one fails, send the exact question text; it gets fixed

with a comment or an example, not with code. Second job: capture screenshots of the identical question asked as advisor, as dean, and as student. Those three screenshots are the fallback if the live demo dies on stage. Third: you hold the Q&A cue cards during judging and feed answers to whoever is speaking."

What the team needs from every member the night before: exact Databricks login email in lowercase; confirmation that "Verify identity" is done on all four accounts; confirmation that OULAD is downloaded and `studentVle` has been uploaded end to end at least once; and confirmation from all three that they have read sections 1-4 of the team brief.

7 · Failure modes and the pre-agreed response

If this happens	This happens next, without a team debate
studentVle upload times out	Split into 4 parts or load gzipped. Decided at 09:00, not at 11:00.
Genie unavailable in Free Edition	Fall back to a Databricks SQL dashboard with the same six questions as saved queries, and lead the pitch with the governance layer. This gets known the night before, because it changes the deck.
Row filter returns zero rows for everyone	Restart the warehouse. Then check <code>current_user()</code> output against <code>role_map</code> for a case mismatch. Then check the EXECUTE grant on the function.
Model won't converge by 13:30	Logistic regression. No discussion, no extension.
Compute quota hit on one account	Switch to another verified account. Four accounts, four budgets — that is the redundancy.
Live demo fails on stage	Kirtee's screenshots and Satyam's screen recording. Narrate over them and keep the pace.

8 · Demo running order — 3 minutes

0:00-0:25	Ojash	The advisor, week six, cannot answer which students are drifting. The data sits in three systems they cannot query.
0:25-1:10	Ojash	The question typed into Genie live. Students named, signals behind each flag, attendance margin left.
1:10-1:35	Ojash	One student's buffer. Stated plainly: this is arithmetic, not prediction. It cannot be wrong.
1:35-2:30	Ojash	The governance moment. Same question, dean's login. Names masked, aggregates appear. The words that get said: Unity Catalog, at the data layer, not the UI, so it cannot be bypassed by querying directly.
2:30-3:00	Satyam	Honesty slide. Attendance synthetic and labelled. Risk model on real OULAD outcomes, weeks 1-6, behaviour only, no demographics. One line on the institutional schema.

Minute 1:35 to 2:30 gets rehearsed harder than the rest combined. It is the only part of this demo other teams cannot also show.